Name: CHUM SOPHEAK

ID: e20241036

Python OOP Report

1. Python Basic Data Types
    a) Primitive / Built-in Data Types

    Python has four basic scalar types: int (arbitrary precision, no overflow), float (64-bit double), bool (True/False, a subclass of int), and str (immutable Unicode text). There are no separate long, char, or double types. They form the core building blocks of any program. Python handles memory dynamically, so you don't need to declare types manually.

    b) Collection and Reference Types

    - list: ordered, mutable, duplicate [1, 2, 3]
    - tuple: ordered, immutable, duplicate (1, 2, 3)
    - dict: mutable, key-value mapping, {‘a’: 1, ‘b’: 2}
    - set: unordered, unique items, {1, 2, 3}
    - objects: instances of user-defined classes

      They allow you to structure and manipulate complex data efficiently without external libraries. The mutable/immutable split: lists and dictionaries are mutable, sets and tuples are immutable, and custom objects are mutable by default. It also explains why plain lists are a poor way to model a record. In its employee example, a missing age silently shifts every later index, and kirk[0] says nothing about what index 0 means. Classes fix both problems.

    c) Literals in Python

    Raw data values written directly into the code and they provide an immediate way to assign constant values to variable.

```
Literals in Python

42, 0b1010, 0o17, 0xFF, 1_000_000   # int: decimal, binary, octal, hex, underscores
3.14, 2.5e3                         # float
'text', 'it is', '''multi-line'''   # string
True, False                         # Boolean
None                                # absence of a value (Java's null)
[1, 2], (1, 2), {'k': 'v'}, {1, 2}  # collection literals
```

Practical example:

```
Practical Example: storing student names, ages, scores, and student records.

name, age, score, enrolled = 'Sokha', 20, 87.5, True

names = ['Sokha', 'Dara', 'Neary']                    # list
ages = (20, 21, 19)                                   # tuple
record = {'name': name, 'age': age, 'score': score}   # dict: one student record
courses = {'Math', 'Physics', 'Math'}                 # set: duplicate removed

class Student:                                        # object: best fit for a record
    def __init__(self, name, age, score):
        self.name, self.age, self.score = name, age, score

s = Student('Sokha', 20, 87.5)
print(record['name'], s.name)
```

2. Python Variables and Attributes

   a) Local Variables

   A variable assigned inside a function or method is local to that call. There is no declaration keyword and no type; assignment creates the variable. They isolate data inside a scope, preventing functions from accidentally overriding each other's data.

   b) Instance Attributes

   Instance attribute is created with self.attribute = value, usually inside .__init__(). Its value belongs to one specific instance and defines that object's state. The book recommends defining all instance attributes in .__init__() so every new object has consistent values. Instance attributes cannot be reached through the class: Car.make raises AttributeError.

c) Class Attributes

A class attribute is a variable assigned directly in the class body, outside any method. All instances share it, so changing it affects every object. The key lookup rule: Python searches the instance's .__dict__ first, then the class's .__dict__.

About the classic pitfall self.num_instances += 1 does not modify the shared class attribute; it creates a new instance attribute that shadows it. To modify class state, go through the class, ideally with type(self).num_instances += 1 so the class name is not hard-coded.

Practical example:

Practical Example: a Student class with individual names and a shared school name.

```
class Student:
    school_name = 'Royal University'          # class attribute (shared)

    def __init__(self, name, age):
        self.name = name                      # instance attributes (individual)
        self.age = age

    def describe(self):
        label = f'{self.name} ({self.age})'   # local variable
        return f'{label} - {type(self).school_name}'

a, b = Student('Sokha', 20), Student('Dara', 21)
Student.school_name = 'City College'          # changes it for both
print(a.describe(), '|', b.describe())

a.school_name = 'Other'                       # creates an instance attribute on a only
print(a.school_name, b.school_name, a.__dict__)
```

3. Python modifiers and Access Control

a) Public, Non-Public, and Private Naming Conventions

Naming rules that signal how a variable or method should be accessed.

- name: public, part of the class's interface
- _name: non-public by convention (internal use)
- __name: private-style, triggers name mangling

To protect internal object states and hide implementation details from outside code.

b) Access Control by convention and Name Mangling

Python enforces nothing for single underscores; it is a signal to other programmers. A double leading underscore makes Python rewrite __balance to _BankAccount__balance. This prevents accidental clashes in subclasses, but it is not real security because the mangled name is still reachable.

c) The @staticmethod Decorator

A static method takes neither self nor cls, so it can touch neither instance nor class state. It is essentially a regular function placed in the class's namespace. It signals independence from everything else, and it is easy to unit test without building an instance.

d) The @classmethod Decorator

A class method receives the class as cls. It can change class-wide state and is the standard way to write alternative constructors (factory methods). Calling cls(...) instead of the class name keeps the code correct if the class is renamed or subclassed.

Practical example:

Practical Example: controlling access to a BankAccount class and defining class-level method

```
class BankAccount:
    bank_name = 'Demo Bank'
    _account_count = 0                          # non-public class attribute

    def __init__(self, owner, balance=0):
        self.owner = owner                      # public
        self._branch = 'Main'                   # non-public by convention
        self.__balance = balance                # name-mangled
        type(self)._account_count += 1

    def deposit(self, amount):
        if not BankAccount.is_valid_amount(amount):
            raise ValueError('Amount must be positive')
        self.__balance += amount

    def get_balance(self):
        return self.__balance

    @staticmethod
    def is_valid_amount(amount):                # no self, no cls
        return amount > 0

    @classmethod
    def from_string(cls, text):                 # alternative constructor
        owner, balance = text.split(',')
        return cls(owner, float(balance))

    @classmethod
    def total_accounts(cls):
        return cls._account_count

acct = BankAccount.from_string('Dara,500')
acct.deposit(100)
print(acct.get_balance(), BankAccount.total_accounts())   # 600.0 1
# print(acct.__balance)                   # AttributeError (mangled)
print(acct._BankAccount__balance)         # 600.0, still reachable
```

e) Abstract Classes and Methods

The abc module provides ABC and @abstractmethod. A class with unimplemented abstract methods cannot be instantiated, and a subclass must override every abstract method to become concrete.

f) Access Control and Encapsulation

Encapsulation in Python means bundling data with the methods that act on it (the source describes this as encapsulating related data and behaviors in a

single entity) and exposing a deliberate interface. Since access is not enforced, encapsulation is achieved through naming conventions, validation in methods, and properties.

g) Other OOP Features

- @property turns a method into an attribute-style accessor, so validation can be added later without changing how callers use the class. A @name.setter makes it writable.
- @final marks constant that should not be reassigned.
- Typing.final marks a method as not overridable, or a class as not subclassable. It is checked only by static tools such as mypy; Python does not enforce it at runtime. There is no final keyword, and @final requires the import.

Practical example:

Practical Example: an abstract Shape class with Circle and Rectangle subclasses.

```python
import math
from abc import ABC, abstractmethod
from typing import final, Final

MAX_DIMENSION: Final = 1000

class Shape(ABC):
    @abstractmethod
    def area(self): ...

    @abstractmethod
    def perimeter(self): ...

    @final                                      # subclasses should not override
    def describe(self):
        return (f'{type(self).__name__}: area={self.area():.2f}, '
                f'perimeter={self.perimeter():.2f}')

class Circle(Shape):
    def __init__(self, radius):
        self.radius = radius                    # goes through the setter

    @property
    def radius(self):
        return self._radius

    @radius.setter
    def radius(self, value):
        if not 0 < value <= MAX_DIMENSION:
            raise ValueError('Invalid radius')
        self._radius = value

    def area(self):
        return math.pi * self._radius ** 2

    def perimeter(self):
        return 2 * math.pi * self._radius

class Rectangle(Shape):
    def __init__(self, width, height):
        self.width, self.height = width, height

    def area(self):
        return self.width * self.height

    def perimeter(self):
        return 2 * (self.width + self.height)

for shape in (Circle(2), Rectangle(3, 4)):
    print(shape.describe())                     # polymorphism
# Shape()                                       # TypeError: abstract class
```

4. Python Operators

a) Arithmetic Operators

The arithmetic operators in python, there are some special methods:

+ (__add__), - (__sub__), * (__mul__), / (__truediv__), // (__floordiv__), % (__mod__), ** (__pow__). Symbols used to perform mathematical calculations. To manipulate numeric state, calculate steps, and process raw metrics.

Arithmetic Operators

```
# Floor division vs standard division
print(5 / 2)   # Output: 2.5 (Float division)
print(5 // 2)  # Output: 2   (Floor division)

# Exponentiation (Power)
print(2 ** 3)  # Output: 8   (2 cubed)
```

b) Comparison Operators

==, !=, >, <, >=, <= map to __eq__, __ne__, __gt__, __lt__, __ge__, __le__. Operators that compare two values and evaluate to a Boolean result (True or False). To drive decision-making pathways and business logic boundaries.

Comparison Operators

```
age = 20
# Operator chaining
is_eligible = 18 <= age <= 25
print(is_eligible)  # Output: True
```

c) Bitwise Operators

Operators that perform actions directly on the binary representation (bits) of integers. Essential for low-level optimizations, network protocols, system flags, and encryption masks.

Bitwise Operators

```
# Bitwise AND (&) and OR (|)
x = 5  # Binary: 0101
y = 3  # Binary: 0011

print(x & y)  # Output: 1 (Binary: 0001)
print(x | y)  # Output: 7 (Binary: 0111)
```

d) Logical Operators

Boolean operators used to combine multiple conditional expressions. To build complex validation guardrails and evaluation filters. and, or, not (not &&, ||, !). They short-circuit and return one of their operands rather than strictly True/False: '' or 'default' gives 'default'.

Practical example:

Practical Example: calculating student grades and checking eligibility.

```
scores = [78, 91, 64]
average = sum(scores) / len(scores)            # true division
bonus = 5 if average > 80 else 0
final_mark = average + bonus

attendance = 92
eligible = final_mark >= 50 and attendance >= 75 and not (final_mark < 0)
honours = 85 <= final_mark <= 100              # chained comparison

print(f'{final_mark:.1f}', eligible, honours)
print(7 // 2, 7 % 2, 2 ** 5)                   # 3 1 32

flags = 0b0101                                 # bitwise: permission-style flags
print(bool(flags & 0b0100), flags | 0b0010)    # True 7
```

e) Assignment Operators

Shortcuts that combine an arithmetic operation with a value assignment. Reduces syntax redundancy and clarifies state mutations.=, +=, -=, *=, /=, //=, %=, **=. The += maps to __iadd__ and similar methods, which should work in place for mutable types. It also shows that self.x += 1 is shorthand for self.x = self.x + 1, which is the cause of the class-attribute shadowing pitfall.

Assignment Operators

```
count = 10
count += 1  # Increments count by 1
print(count)  # Output: 11
```

f) Special Operators

- Identity: is / is not test whether two names refer to the same object. Use == for value equality and is mainly for None checks (x is None).
- Membership: in / not in are supported by __contains__. The source implements it for a stack class and notes that lists support it by default.
- Conditional expression: x if condition else y.

Special Operators

```
# Membership testing
fruits = ["apple", "banana", "cherry"]
print("apple" in fruits)  # Output: True

# Identity testing
list1 = [1, 2]
list2 = [1, 2]
print(list1 == list2)  # Output: True (Values are identical)
print(list1 is list2)  # Output: False (Distinct memory references)
```

g) Operator Precedence and Associativity

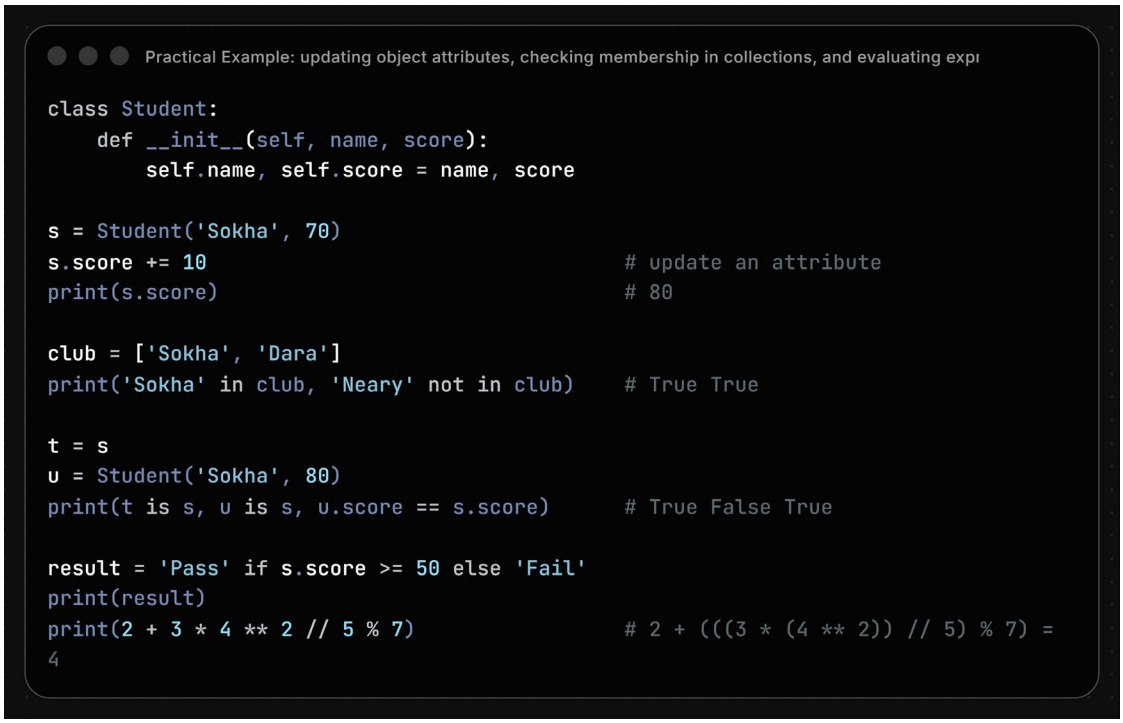
The predefined rules determining execution sequence when multiple operators appear in a single expression. Prevents logical ambiguity and yields deterministic processing sequences.

Operators Precedence and Associativity

```
# Exponentiation right-to-left associativity: 2 ** (3 ** 2) -> 2 ** 9
print(2 ** 3 ** 2)  # Output: 512

# Standard Precedence: Multiplication before addition
print(5 + 3 * 2)  # Output: 11
```

Practical example:



5. Python Decision-Making Statements

   a) The if Statement

      A basic structural gate that runs an execution block only if its condition resolves to True. Isolates optional paths based on dynamic evaluations.

   b) The if-else Statement

      A binary branching construct providing an alternative path when the if condition evaluates to False. Ensures a default execution block runs when primary checks fail.

   c) The if-elif-else Statement

      A sequential evaluation ladder checking multiple expressions from top to bottom. Streamlines multi-option triage without forcing massive deeply-nested indents.

      Practical example:

Practical Example: determining student pass/fail status and assigning letter grades.

```
score = 73

if score >= 50:                        # a. if
    print('Pass')

if score >= 50:                        # b. if-else
    status = 'Pass'
else:
    status = 'Fail'

if score >= 90:                        # c. if-elif-else
    grade = 'A'
elif score >= 80:
    grade = 'B'
elif score >= 70:
    grade = 'C'
elif score >= 50:
    grade = 'D'
else:
    grade = 'F'

print(status, grade)                   # Pass C
```

d) Nested if Statements

Nested if Statements means if inside another if. Handles sequential multi-layered check dependencies before executing target operations.

Nested if statements

```
if attendance >= 75:
    if score >= 50:
        print('Eligible')
# equivalent: if attendance >= 75 and score >= 50:
```

e) Multiple-Branch Selection Using match-case or if-elif-else

A control flow structure designed to evaluate an expression against discrete literal matching rules or structural schemas. It offers an incredibly clean alternative to long, repetitive if-elif statements when evaluating a single clear variable choice.

Multiple-Branch Selection using match-case

```
# Menu selection for an academic records database
menu_choice = "2"

match menu_choice:
    case "1":
        print("Routing to: Register New Student Profiles")
    case "2":
        print("Routing to: Fetching Existing Student Records")  # Executes
here
    case "3":
        print("Routing to: Termination Protocol")
    case _:  # Behaves like 'default:' in Java switch blocks
        print("Exception: Unrecognized Navigation Request")
```

f) Conditional Expressions

A compact inline statement that evaluates a condition and returns one of two values on a single line. Keeps code tidy and concise when assigning values to a variable based on simple binary conditions.

Conditional Expressions

```
score = 68

# Pythonic single-line conditional assignment
status = "Pass" if score >= 50 else "Fail"
print(f"Final Class Status: {status}")
```

Practical example:

Practical example: a menu-driven student management system.

```
students = {}                                   # name -> score

while True:
    print()
    print('1) Add  2) View  3) Find  4) Delete  0) Exit')
    choice = input('Choice: ').strip()

    match choice:
        case '1':
            name = input('Name: ').strip()
            try:
                score = float(input('Score: '))
            except ValueError:
                print('Score must be a number.')
            else:
                if 0 <= score <= 100:
                    students[name] = score
                    print('Added.')
                else:
                    print('Score must be 0-100.')
        case '2':
            if not students:
                print('No students yet.')
            for n, s in students.items():
                label = 'Pass' if s >= 50 else 'Fail'
                print(f'{n}: {s} ({label})')
        case '3':
            name = input('Name: ').strip()
            print(students.get(name, 'Not found'))
        case '4':
            name = input('Name: ').strip()
            removed = students.pop(name, None)
            print('Deleted.' if removed is not None else 'Not found.')
        case '0':
            print('Goodbye!')
            break
        case _:
            print('Invalid choice.')
```

6. Python Loops and Loop Control

   a) The while Loop

      Runs while a condition is true: while count < 3:.

```
while-loop

count = 1
while count <= 3:
    print(count)
    count += 1
```

b) The for Loop

Python's for always iterates over an *iterable* (list, string, range(), dictionary, and so on), like Java's for-each loop.

```
for-loop

names = ["Ana", "Ben"]
for i in range(1, 4):
    print(i)
for name in names:
    print(name)
```

c) Repetition with for and while

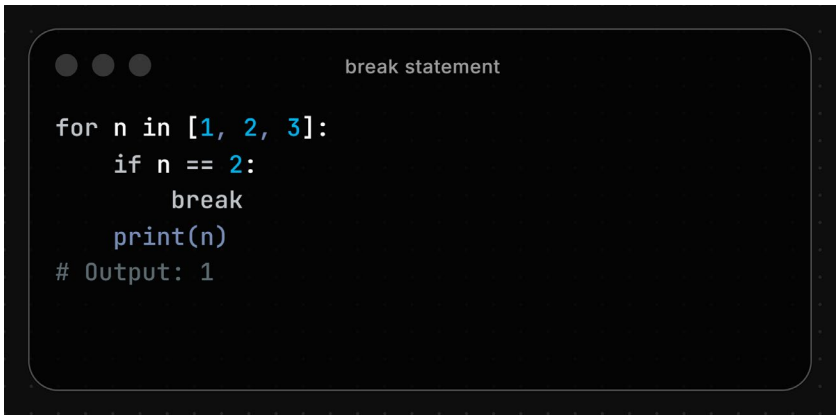
Use for when you know what you are iterating over, and while when you repeat until a condition changes. Python has no do-while; emulate it with while True: and a break at the end of the body.

```
do-while-loop with break statement

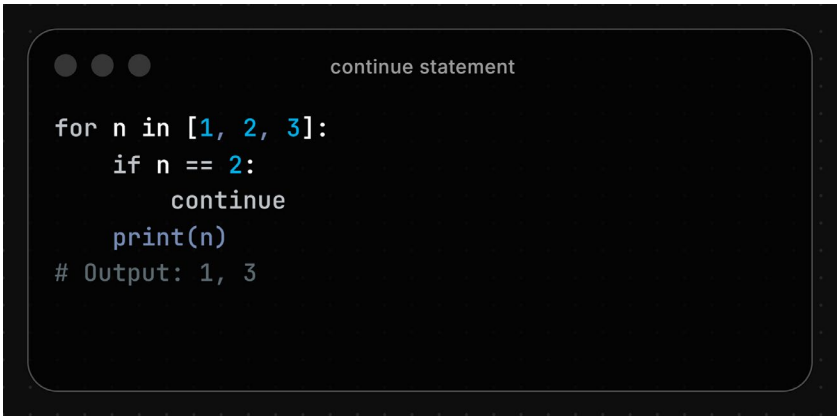
while True:
    n = int(input())
    if n > 0:
        break
```

d) Loop Control Statements

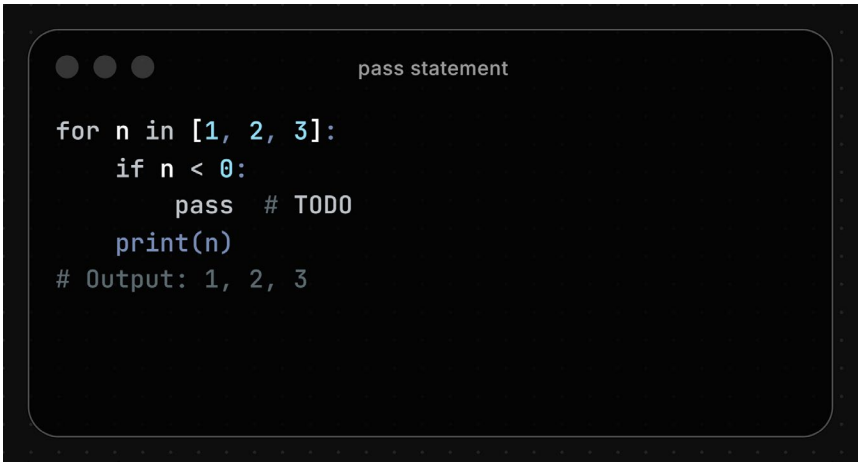
- break exits the nearest enclosing loop immediately (items d and e in the outline cover the same statement).



- continue skips the rest of the current iteration.



- pass does nothing and is a placeholder where a statement is syntactically required. The source uses it in empty class bodies such as class Dog: pass.



- A loop may also have an `else` clause that runs only if the loop finished *without* hitting `break`.

Practical example:

Practical example: iterating through student objects, calculating averages, and searching for a student

```
class Student:
    def __init__(self, name, score):
        self.name, self.score = name, score

    def __repr__(self):
        return f'Student({self.name!r}, {self.score})'

students = [Student('Sokha', 87), Student('Dara', 45),
            Student('Neary', 92), Student('Vibol', 70)]

total = 0
for s in students:                              # iterate over objects
    total += s.score
print('Average:', total / len(students))        # 73.5

target, found = 'Neary', None
for s in students:                              # search with break / for-else
    if s.name == target:
        found = s
        break
else:
    print('No such student')
print(found)

for i, s in enumerate(students, start=1):       # continue: skip failures
    if s.score < 50:
        continue
    print(i, s.name)

attempts = 0                                    # do-while emulation
while True:
    attempts += 1
    if attempts >= 3:
        break
print(attempts)
```